TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG – ĐHBK HÀ NỘI

BỘ MÔN KHOA HỌC MÁY TÍNH & HỆ THỐNG THÔNG TIN

CẨM NANG HƯỚNG DẪN THỰC HIỆN ĐỒ ÁN 2

Đề tài: Hệ Thống AI Agent Hỗ Trợ Tra Cứu và Quản Lý Vật Tư - Thiết Bị Phòng Ban

---

CHUYÊN ĐỀ TUẦN 03 (DÀNH CHO NGƯỜI MỚI BẮT ĐẦU TỪ CON SỐ 0)

BẢN CHẤT AI AGENT, CHỐNG ẢO GIÁC SỐ HỌC & NHẬP MÔN LANGGRAPH

Sinh viên thực hiện: Trần Huy Hoàng | **MSSV:** 20237337

Đơn vị áp dụng thử nghiệm: Phòng Hành chính – Quản trị (Ban Quản lý Cơ sở vật chất)

Thời gian học tập: Tuần 3 (Khám Phá Não Bộ Điều Phối AI Agent & LangGraph)

---

1. Mục Tiêu Tuần 03: Vượt Qua Sự Nhầm Lẫn Giữa Chatbot & Agent

Chào Hoàng, sau khi hoàn thành 2 tuần đầu, em đã có một nền tảng dữ liệu SQLite rất tốt. Tuy nhiên, khi nói chuyện với các bạn sinh viên khác, thầy thấy đa số các em đều nhầm lẫn: *"Ủa, em thấy hệ thống này cũng có một khung chat gõ câu hỏi vào rồi nó trả lời, vậy nó khác gì con ChatGPT hay mấy con bot trả lời tự động trên Fanpage Facebook?"*

Đây là câu hỏi mà **100% các thầy cô trong Hội đồng chấm thi sẽ hỏi em**. Nếu không trả lời sắc bén, đồ án sẽ bị đánh giá là tầm thường. Vì vậy, mục tiêu của Tuần 03 là:

1. Phân biệt bản chất 3 thế hệ công nghệ: Chatbot cũ → LLM thông thường → AI Agent.
2. Hiểu tại sao các mô hình AI lớn lại dốt toán và bài báo khoa học quốc tế **arXiv:2602.12631 (2026)** giải quyết hiện tượng **Ảo giác tính toán (Hallucination Tax)** như thế nào.
3. Nhập môn framework điều phối hiện đại nhất hiện nay: **LangGraph** dưới lăng kính mạng lưới tàu điện ngầm.

2. Ba Cấp Độ Tiến Hóa Của Giao Diện Hội Thoại

Để hiểu AI Agent, em hãy so sánh 3 thế hệ công nghệ sau:

Đặc Điểm	Cấp Độ 1: Chatbot Cũ	Cấp Độ 2: ChatGPT Thường	Cấp Độ 3: AI Agent (Đề Tài Này)
Cơ chế	Dựa trên từ khóa cứng hoặc cây `if/else` cố định.	Mô hình ngôn ngữ lớn (LLM). Dự đoán từ ngữ tiếp theo.	Mô hình trạng thái kết hợp công cụ tác vụ (Stateful Tools).
Khả năng hành động	Không thể làm gì ngoài các câu thoại viết sẵn.	Chỉ sinh văn bản (text) trên màn hình, không chạm được vào máy tính.	**Tự động thực thi hành động:** Truy vấn DB, cào web, sinh file Word, vẽ biểu đồ.
Xử lý bài toán	Hỏi lệch 1 chữ là "Xin lỗi tôi không hiểu".	Hỏi 1 câu trả lời 1 lần (Single-turn prompt).	**Lập kế hoạch đa bước:** Hiểu ý định → Soát vé → Chọn tool → Cập nhật CSDL.

Bảng 1: So sánh 3 cấp độ từ Chatbot đến AI Agent

Ví dụ đời thường giúp em trả lời tự tin trước Hội đồng

Giả sử có nhân viên nói: *"Máy in tầng 2 bị kẹt giấy, hãy lập phiếu báo hỏng giúp tôi"*.

- **ChatGPT thông thường:** Sẽ viết ra một đoạn văn bản mẫu trên màn hình và bảo người dùng: *"Bạn hãy copy đoạn này rồi dán vào Microsoft Word nhé"*.
- **AI Agent của em:** Tự động trích xuất tên người dùng, phòng ban, dòng máy Canon 2900 → gọi module `doc_generator.py` → tạo ra file `Phieu_Bao_Hong_SC-959E91.docx` thật sự trên ổ cứng → lưu vào bảng `maintenance_tickets` → trả về nút tải file ngay trên khung chat!

3. Hiện Tượng Ảo Giác Của LLM & Bài Báo arXiv:2602.12631 (2026)

3.1. Tại sao mô hình ngôn ngữ lớn lại "dốt toán"?

Các mô hình như GPT-4 hay Gemini được huấn luyện dựa trên kiến trúc Transformer với mục tiêu: **Dự đoán token (từ ngữ) tiếp theo có xác suất xuất hiện cao nhất**. Bộ não của chúng là một mạng nơ-ron xác suất ngôn ngữ, **không phải là một chip số học ALU**.

Khi người dùng hỏi: *"Kho có 85 ram giấy, xuất đi 18 ram cho Marketing, xuất tiếp 23 ram cho Kế toán, rồi nhập về 40 ram, tổng cộng còn bao nhiêu?"*.

- Đôi khi mô hình sẽ trả lời một con số sai lệch hoàn toàn vì nó đang "đoán từ" theo cảm giác ngôn ngữ.
- Trong quản lý kho và tài sản doanh nghiệp, sai 1 đơn vị hàng là lệch sổ sách kế toán hàng chục triệu đồng!

3.2. Công bố khoa học arXiv:2602.12631: "Hallucination Tax"

Bài báo khoa học quốc tế mới nhất năm 2026 mang tên:

> *"AI Agents for Inventory Control: Human-LLM-OR Complementarity"* (**arXiv:2602.12631**, 2026)

đã chỉ ra rằng: Nếu để LLM tự tính toán số học tồn kho, doanh nghiệp sẽ phải trả một chi phí vô cùng đắt đỏ gọi là **"Hallucination Tax" (Thuế ảo giác)**.

3.3. Giải pháp của đồ án: Tính toán số học tất định (Deterministic Code Execution)

Để triệt tiêu hoàn toàn lỗi này, hệ thống của em áp dụng phương pháp phân công lao động khoa học:

- **AI:** Chỉ làm việc nó giỏi nhất – đó là **Hiểu ngôn ngữ tự nhiên tiếng Việt** của nhân viên (Nhân viên đang hỏi gì? Đang cần mượn đồ hay báo hỏng?).
- **Code Python & SQL:** Đảm nhận toàn bộ việc cộng trừ tồn kho, tính tổng định giá tài sản ($85 \times 75.000 = 6.375.000$ VNĐ) và kiểm tra chặn âm kho.
- **Kết quả:** Độ chính xác số học đạt **100% tuyệt đối**, loại bỏ hoàn toàn Hallucination Tax!

4. Nhập Môn LangGraph: Mô Hình Tuyến Tàu Điện Ngầm

Để liên kết các hành động lại với nhau, đồ án sử dụng framework **LangGraph**. Hãy hình dung LangGraph giống như một **Tuyến Tàu Điện Ngầm**:

1. **AgentState (Toa tàu mang hành lý):** Là một từ điển Python (`TypedDict`) mang theo toàn bộ dữ liệu của lượt tương tác hiện tại: câu hỏi người dùng, vai trò (`user`/`admin`), ý định (`intent`), văn bản trả lời (`response`), và dữ liệu đính kèm.
2. **Node (Nhà ga xử lý):** Mỗi Node là một hàm Python nhận vào State, thực hiện một nhiệm vụ (tra cứu kho, hướng dẫn kỹ thuật, cào web), cập nhật thêm dữ liệu vào State rồi chuyển toa tàu đi tiếp.
3. **Edge (Đường ray cố định):** Đường nối thẳng từ Node A sang Node B.
4. **Conditional Edge (Bộ bẻ ghi đường ray có điều kiện):** Dựa trên ý định và quyền hạn của người dùng để quyết định toa tàu rẽ sang nhánh nào.

Định nghĩa State trung tâm trong ‘src/agents/orchestrator.py‘

```python
from typing import Dict, Any, TypedDict, Optional

class AgentState(TypedDict):
    user_message: str            # Cau hoi tieng Viet cua nguoi dung
    user_role: str               # 'user' hoac 'admin' (RBAC)
    user_name: str               # Ten can bo / nhan vien
    department: str              # Phong ban
    image_source: Optional[Any] # Anh hoa don (neu co)
    intent: str                  # Y dinh sau phan loai
    response: str                # Cau tra loi cuoi cung
    data: Dict[str, Any]         # Du lieu phu tro (duong dan file Word, anh chart
        ...)
```

Bộ bẻ ghi rẽ nhánh có điều kiện (‘route_intent‘)

```python
from langgraph.graph import END

def route_intent(state: AgentState) -> str:
    intent = state.get("intent", "inventory")
    if intent == "blocked":       return END                # Bi chan quyen -> Dung
        ngay
    elif intent == "ocr_restock": return "ocr_node"         # Nhay sang doc anh hoa
        don
```

```
    elif intent == "enrich":      return "enrich_node"     # Nhay sang cao web cam nang
    elif intent == "support":     return "support_node"    # Nhay sang tro ly sua loi
    else:                         return "inventory_node"  # Mac dinh tra cuu kho
```

5. Bài Tập Rèn Luyện Tuần 03 Dành Cho Sinh Viên

Hoàng hãy ghi nhớ và thực hành 2 câu hỏi trọng tâm sau:

1. **Câu 1 (Luyện thi vấn đáp):** Khi thầy cô hỏi: *"Tại sao em không dùng luôn chuỗi if/else cho gọn mà phải dùng LangGraph?"*, em sẽ trả lời thế nào? (Gợi ý: Nhắc đến khả năng mở rộng thêm các Agent con, cơ chế lưu trạng thái State, và khả năng tạm dừng luồng chờ người bấm duyệt Human-in-the-Loop).
2. **Câu 2 (Thực hành code):** Viết một hàm Python đơn giản nhận vào tin nhắn của người dùng và trả về `intent = "support"` nếu câu hỏi chứa các từ: *"hỏng"*, *"không lên"*, *"kẹt giấy"*, *"no signal"*.

Thầy chúc mừng em đã hiểu được bộ não LangGraph! Sang Tuần 04, chúng ta sẽ bắt tay vào chế tạo **Bộ công cụ (Tools)**: cào web, vẽ biểu đồ và tự động sinh file Word!